



CAESAR'S GAMBIT

STRATEGISCHES KRIEGSSPIEL

Svenja Adam • Matthias Hauswirth • Simon Knöpfler
Justin Schuler • David Frommann

Agenda

01 Business Case & Projektvision

02 Projektmanagement

03 Architektur & Tech-Stack

04 Designentscheidungen

05 Live-Demo

06 Tests, Metriken & Qualität

07 CI/CD Setup

08 Herausforderungen & Learnings

09 Rückblick

10 Q&A



01 (Business Case &) Projektvision & Ziel



Vision: Browser-basiertes Risiko-ähnliches Spiel



Ziel: sofort spielbar, kein Download nötig, einfaches zusammen Spielen mit Freunden



Zielgruppe: Studierende, Freunde, kleine Gruppen (3–6)



Mehrwert des Spiels: Sozialkontakte, Kindheitserinnerungen, altes Spiel in neuem Look



Mehrwert für uns: Verbindung von Spielspaß, moderner Web-Technologie und sauberem Software-Engineering

02 Projektmanagement-Statistiken

- ▶ Scrum mit 2-Wochen-Sprints, Sprint-Planung in Jira.
- ▶ Wöchentliche Blogbeiträge via GitHub Discussions.
- ▶ Dailys montags & mittwochs (15 min), Weekly bei bedarf (2 h).

Agile Manifesto - Reflektions- & Polishing-Prinzip



03 Architektur



- ▶ **Frontend:** reaktive schnelle UI, klare Komponentenstruktur.
- ▶ **Backend:** zentrale Spiellogik im Model, Validierung aller Züge.
- ▶ **Schichtenarchitektur:** Controller → Service → Model → Repository.
- ▶ **Kommunikation:** SSE (Server-Sent Events) für Echtzeit-Updates.
- ▶ **Datenbank:** für persistente Nutzerdaten.
- ▶ **klickbare Karte.**



03 Tech-Stack

PROJECT & TOOLS



Jira

Issue Tracking,
Agile Management



Git & GitHub

Version Control,
Repository



Docker

Containerization,
Deployment



Google Chrome

Target Browser,
Development

LANGUAGES & FRONTEND STACK



TypeScript

Strongly Typed JS



React

UI Component Library



Next.js

React Framework



Tailwind CSS

Utility-First CSS



tanstack

React Query, Table, etc.



react-dice-complete

Game Component

BACKEND & DATA



Java

Application Language



Spring Boot

Backend Framework



JWT JSON Web Tokens



JUnit Java Testing Framework



PostgreSQL

Persistent Data Storage

METRICS & QUALITY TOOLS



SonarQube

Static Code
Analysis



Ck Metrics

Software Metrics



eslint

JS/TS Linting

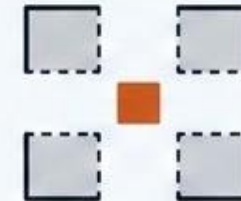


vitest

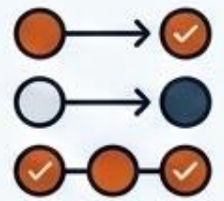
TS Testing

A diagram illustrating the relationship between Shadcn Components and TailwindCSS. It features two white rounded rectangular boxes. The left box contains the Shadcn Components logo (a blue cube icon) and the text "Shadcn Components". An arrow points from this box to the right box, which contains the TailwindCSS logo (two teal wavy lines) and the text "TailwindCSS". Below these boxes, the text "Konsistentes, modernes UI." is written in a bold, black font. At the bottom of the slide, there are three overlapping visual elements: a dark-themed code editor window on the left displaying JSX code for a comment component, a smaller version of the same code editor in front of it, and two light-colored mobile app mockups on the right showing user profile and comment sections.

Kenemistnonden Irs Nähe
und Nähe anézu keinen Nähe.



Useranstenefirithin Kōiiso
anterunige und Weißraum.



Konsistenz von Konsisten und
Konsistenz und Konsistenz.



Choize

Choice 2

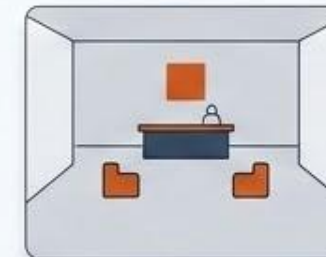
Aktionsauswahl – Entscheidung

Userverwaltung



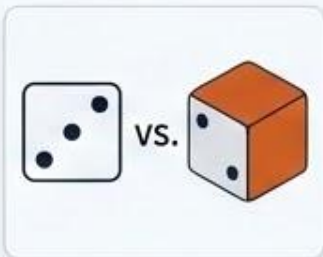
Einmal-Implementations die
Soheit der Userverwaltung.

Lobby



Unf. hterwertugen Implementierung
die Katsromersatorung.

Würfeln



Abstract Addülierung und
Features finaleandüierung.

Risiko online

create room

room id

join room

Risiko online #room-id

 Spieler 2 won

 Spieler 1

 Spieler 2

 Spieler 3

 Spieler 4

Start new round

delete room

 share room link

Caesar's Gambit

Hauptmenü

Willkommen, matias!

Erstelle eine Lobby, tritt einer Partie bei oder passe dein Profil an.

Spielmenü
Wähle deine Aktion.

Spiel erstellen

Lobby beitreten

Einstellungen

Hilf bei der Entwicklung

Abmelden

© 2026 Caesar's Gambit - Strategisches Kriegsspiel
Version 1.0.0

[GitHub](#) | [Datenschutz](#) | [Impressum](#)

Caesar's Gambit

LOBBY

ROOM #1

AKTIVE SPIELER

 matthiasderhund STAB

 test2test2

Spiel Starten

 Einladung kopieren

Raum verlassen

Chat

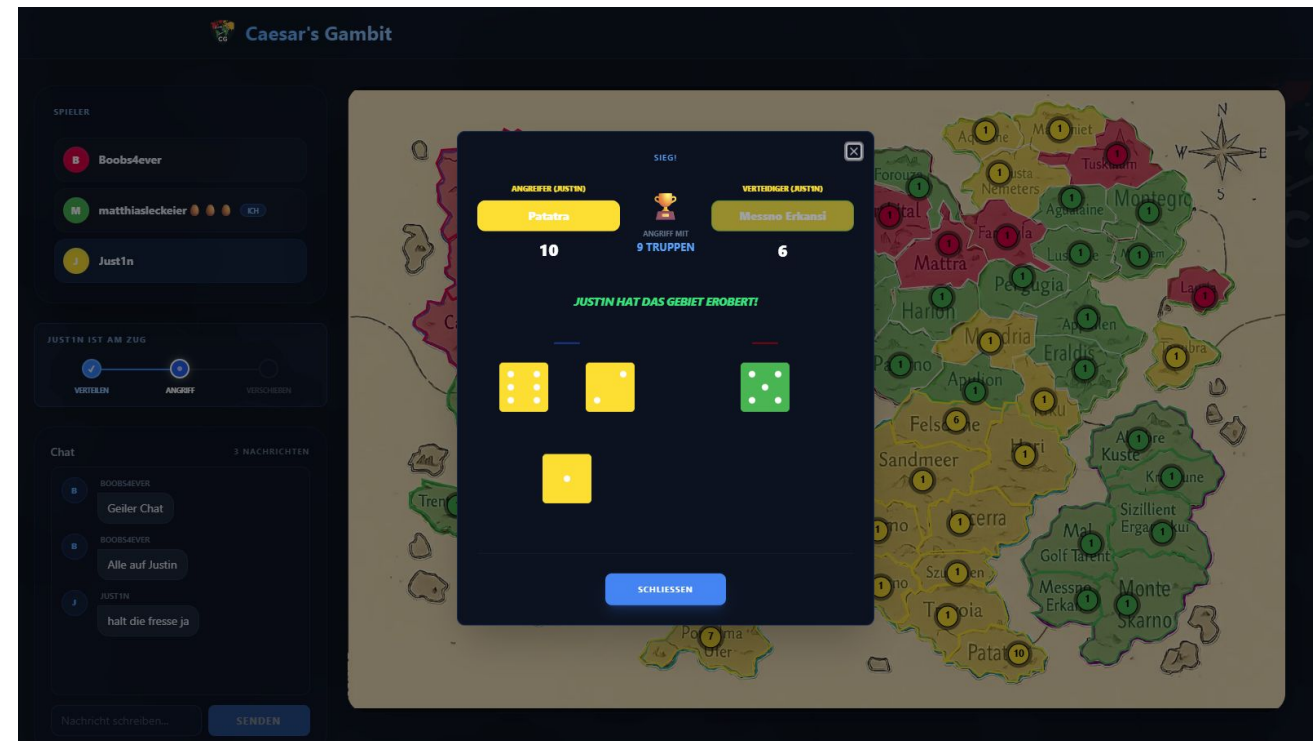
0 NACHRICHTEN

Kein Chatverlauf

Nachricht schreiben...

SENDEN

Risiko online #room-id



05 Live-Demo

2 Zuschauer auswählen die mit joinen und demo mitmachen

1 Login / Registrierung



Nutzer erstellen und anmelden.
dann einmal Benutzernamen-Änderung

2 Lobby



Raum erstellen, zwei Zuschauer
joinen rein

3 Spielstart



Karte mit klickbaren Regionen
wird angezeigt

4 Truppenverteilung



DistributionDialog, Aufstell-Phase

5 Angriff + Bewegung



Zug ausführen, UI reagiert in Echtzeit

6 Chat funktion



7 Backend-Flow



kurz zeigen – wie State gespeichert /
ausgelesen wird

06 Tests, Metriken & Qualität



Test-Ansatz & Fokus

- ▶ Unit- & Component-Tests
- ▶ Fokus: Backend-Logik (Regeln, Validierung)
- ▶ Isoliertes Testen der Spiellogik
- ▶ Frühzeitige Fehlererkennung
- ▶ Frontend-Tests: Ergänzung für UI-Szenarien



Traceability & Transparenz

- ▶ Frontend-Testfälle direkt bei Komponenten
- ▶ GitHub Commit-Historie nachverfolgbar
- ▶ Automatisierte CI/CD Test-Pipelines



Metriken & Qualitäts-Trends

- ▶ Cyclomatic Complexity **9.6 → 11.42 ↗**
- ▶ Maintainability (Code Smells) **30 → 46 ↗**
- ▶ Technical Debt (Hours) **5h16 → 5h55 ↗**
- ▶ CBO (Avg / Max) **5.54 → 6.75 / 19 ↗**
Max CBO Klasse: GameController (unverändert)

Validierung durch externe Tools

File	% StmtS	% Branch	% Funcs	% Lines
All files	62.53	57.37	63.82	63.92

risikotest

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.risiko.config		0 %		0 %	17	17	37	37	16	16	3	3
com.risiko.model		95 %		83 %	25	150	21	536	4	84	0	6
com.risiko.controller		93 %		83 %	11	50	2	128	6	35	0	6
com.risiko.security		96 %		86 %	2	19	2	45	0	8	0	2
com.risiko		0 %		n/a	2	2	3	3	2	2	1	1
com.risiko.services		97 %		85 %	5	30	2	60	1	16	0	2
com.risiko.model.dto		100 %		n/a	0	48	0	93	0	48	0	7
com.risiko.exception		100 %		n/a	0	6	0	10	0	6	0	2
Total	381 of 4.194	90 %	36 of 214	83 %	62	322	67	912	29	215	4	29

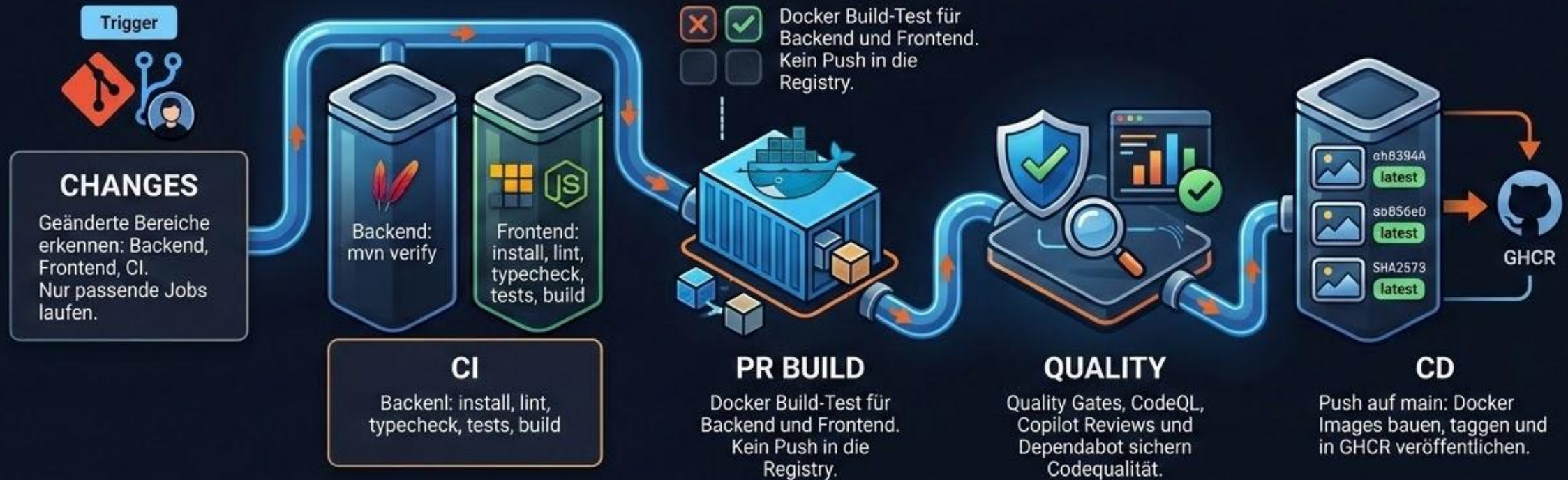
07 CI/CD Setup

CI/CD Pipeline

Automatisierte Qualitätssicherung, Build-Validierung und Container-Auslieferung

Trigger: Push & Pull Request auf main

GitHub Actions · Backend Maven/Java 25 ·
Frontend pnpm/Node 22 · GHCR



Schneller Feedback-Loop

Fehler, Typ-Probleme und fehlgeschlagene Tests werden vor dem Merge sichtbar.



Sicherer Merge-Prozess

Quality Gates und Security-Scans blockieren problematische Änderungen frühzeitig.



Reproduzierbares Deployment

Container Images werden versioniert mit SHA und latest in GitHub Container Registry abgelegt.

08 Herausforderungen & Learnings



Learnings

- ▶ Jira und GitHub Discussions als Kommunikationszentrum.
- ▶ Klare Aufgabenverteilung verhindert Merge-Konflikte.
- ▶ Game-Development-Erfahrung, Multiplayer-Synchronisation, moderne/ modulare Web-Architektur, sauberer Code.
- ▶ Rollenaufteilung flexibler
- ▶ Conventions am Anfang einführen (Tailwind/ pnpm)
- ▶ stabiles Grundgerüst am Anfang aufsetzen (Api client im Frontend)
- ▶ auf Jira Board aufpassen

Herausforderungen

- ▶ Synchronisation bei mehreren Clients
- ▶ **Server-Autorität** als Single Source of Truth.



SVG-Map: Klickbarkeit



- ▶ SVG-Map: Regionen klickbar machen war technisch aufwändig.

09 Rückblick

Umsetzung des Ausblick der Zwischenpräsentation:



Spiel mit allen Kernfunktionen umgesetzt



Namensänderung ist umgesetzt



Umsetzung des Learnings (1 Semester):
Mehr Zeit für Testing einplanen

Leider mussten wir Funktionsumfang verkleinern, damit das Spiel in einer soliden Grundfassung rechtzeitig released werden kann.





Danke für eure Aufmerksamkeit!



Habt ihr noch Fragen?

